

LAB 2

GPIO Interfacing- PART 2

Objectives:

WEEK 6

- Learn how to use GPIO to control **LEDs and 7-Segment Displays** on the **TI TIVA Launchpad** and write the required firmware
- Implement button debouncing to ensure reliable switch inputs.
- Use functions to organize code for better readability and modularity.
- Create an increment/decrement counter using SW1 and SW2.

Material:

- **Keil uVision5 IDE**
- **TIVA C Series LaunchPAD**
- **Common Anode 7-Segment Display** (available in your toolkit)
- **74LS247 IC** (BCD to 7-Segment Decoder/Driver IC, available in your toolkit)
- **Breadboard and Jumper Wires**

Submission Instruction

1. **Demonstrate** your work for each activity to your instructor. Missing the instructor signature results in **20% penalty**.
2. **Submit** a zipped file of your project folder onto the BB dropbox. Missing the zipped project file results in a **20% penalty**.
3. **Write** a lab report detailing the implementation and results. Include the picture of your circuit, screenshot of your code and results. Answer all the questions.

Lab Activity 1- Control on Board LEDs with SW1 and SW2 [40 points]

Objective:

- Write a program that controls Red, Green, and Blue LEDs with SW1 and SW2 with the following pattern.
- Implement **Button Debouncing**
 - Read **SW1** and **SW2** states.
 - If a button is **pressed**, wait **100-200ms** and recheck to confirm the press.

SW1	SW2	Red	Green	Blue
Released	Released	ON	ON	ON
Pressed	Pressed	ON	OFF	OFF
Released	Pressed	OFF	ON	OFF
Pressed	Released	OFF	OFF	ON

LAB Activity 2- Create an Up/Down Counter [40 points]

Objective:

- Connect a 7-segment display using 74LS247 IC as you learned in LAB2-Part 1.
- Write a program that reads the status of SW1 and SW2.
 - When **SW1** is pressed, **increment** the number on 7-segment display.
 - When **SW2** is pressed, **decrement** the number on 7-segment display.
- Important considerations:
 - Implement **Button Debouncing** to prevent false switch reading.
 - The up counter should count to 9 and then reset to 0.
 - The down counter should count down to 0 and then reset to 9.

Questions [20 points]

1. How did implementing **debouncing** improve the reliability of switch inputs?
2. Why is it necessary to configure **pull-up resistors** for SW1 and SW2?
3. How does using **modular functions** improve the readability and maintainability of your code?
4. Suppose you needed to control **external LEDs** instead of the onboard ones. What changes would be required in the circuit and code?
5. If the counter needed to **display numbers beyond 0-9**, what modifications would you make?